

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 1 – Git & Docker Technical Assignment

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 – Git & Docker Technical Assignment
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	G.BHARATHI	Reg. No.:	192371079
Date:	22-08-2026	Duration:	60 minutes

Code of Conduct

I G.BHARATHI 192371079(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:G.BHARATHI

Annexure A – Git & Docker Technical Assignment

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, prepare a **Git & Docker Technical Assignment** by applying version control and containerization concepts. Your report should include appropriate diagrams, screenshots, commands, and explanations wherever applicable.

Assignment Questions

1. Problem Analysis

- Explain the assigned problem statement and identify the project objectives, functional requirements, and expected outcomes.

2. Project Structure Design

- Design and explain the folder structure of your project repository.
- Justify the purpose of each major folder and file included in the repository.

3. Git Repository Initialization

- Create a Git repository for the project.
- Explain the steps involved in repository initialization and describe the purpose of the essential Git files.

4. Git Branching Strategy

- Design a suitable branching strategy (e.g., Main, Development, Feature, Release, Hotfix).
- Justify why the selected branching model is appropriate for the assigned project.

5. Version Control Workflow

- Demonstrate the Git workflow by performing meaningful commits, branch creation, merging, conflict resolution (if applicable), and repository synchronization.
- Explain the purpose of each Git operation performed.

6. Commit History Analysis

- Present the commit history of the project.
- Explain how commit messages follow good version control practices and contribute to project maintenance.

7. Docker Environment Design

- Explain why Docker is suitable for the assigned project.
- Identify the application components that require containerization.

8. Dockerfile Development

- Design and explain the Dockerfile required to containerize the application.
- Describe the purpose of each instruction used in the Dockerfile.

9. Docker Compose Design

- Develop a Docker Compose configuration for the project.
- Explain the services, networking, volumes, environment variables, and dependencies used.

10. Container Deployment and Testing

- Demonstrate the execution of the application using Docker containers.
- Explain the testing procedure and provide evidence that the application runs successfully.

11. Benefits of Git and Docker

- Discuss how Git and Docker improve collaboration, version management, portability, deployment, scalability, and maintainability for your assigned project.

12. Challenges and Improvements

- Identify the challenges encountered during implementation.
- Suggest possible improvements and future enhancements for the Git workflow and Docker deployment.

Expected Deliverables

- Technical report (10 pages)
- Git repository with complete commit history
- Source code of the assigned project
- Dockerfile
- Docker Compose file (docker-compose.yml)
- Screenshots of Git operations and Docker execution
- Architecture and workflow diagrams

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Project Design(20%)	Clearly explains the problem statement, objectives, requirements, expected outcomes, and well-organized	Good analysis and project structure with minor omissions.	Basic analysis and repository structure with limited justification.	Incomplete or unclear analysis and project organization.

	project structure with proper justification.			
Git Repository & Branching Strategy(20%)	Repository initialized correctly with appropriate branching strategy, clear workflow, and proper repository organization.	Repository and branching strategy are mostly correct.	Basic Git setup with limited branching implementation.	Incorrect or incomplete Git repository and branching strategy.
Version Control Workflow & Commit History(10%)	Demonstrates meaningful commits, branching, merging, synchronization, conflict resolution, and professional commit messages with clear history.	Most Git operations demonstrated correctly with good commit history.	Limited Git workflow and commit practices.	Poor workflow and inadequate commit history.
Docker Implementation(20%)	Well-designed Docker environment, Dockerfile, and Docker Compose configuration with clear explanation of services, networking, volumes, and dependencies.	Functional Docker implementation with minor issues.	Basic Docker implementation with limited explanation.	Incomplete or incorrect Docker implementation.
				Deployment

Deployment, Testing & Results(15%)	Application successfully deployed and tested with appropriate screenshots and evidence of successful execution.	Deployment completed with minor issues.	Partial deployment and testing demonstrated.	unsuccessful or insufficient evidence.
Documentation, Challenges & Future Enhancements(15%)	Professional report (10–15 pages) with diagrams, screenshots, references, discussion of benefits, challenges, and future improvements.	Good documentation with minor omissions.	Basic documentation with limited discussion.	Poor documentation and insufficient analysis.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25

Git & Docker Technical Assignment

1. Problem Analysis

1.1 Problem Statement

The College Administration and Management System is designed to manage the major administrative activities of a college through a centralized software system. The system helps administrators manage student details, faculty information, departments, courses, admissions, attendance, fees, and reports efficiently.

The system reduces manual paperwork, minimizes errors, provides quick access to information, and improves the overall efficiency of college administration.

1.2 Project Objectives

The main objectives of the system are:

- To manage student information efficiently.
- To maintain faculty and department details.
- To manage student admissions and enrollment.
- To maintain course and academic information.
- To record and monitor student attendance.
- To manage student fee details.
- To generate administrative reports.
- To reduce manual paperwork and errors.
- To provide centralized and secure data management.
- To simplify college administrative operations.

1.3 Functional Requirements

The major functional requirements of the system are:

Student Management:

The system allows administrators to add, update, view, and manage student information.

Faculty Management:

The system maintains faculty details and their department and course assignments.

Department Management:

Administrators can create and manage department information.

Course Management:

The system allows administrators to add, update, and manage courses.

Admission Management:

The system maintains student admission and enrollment details.

Attendance Management:

The system records student attendance and generates attendance reports.

Fee Management:

The system records fee payments and identifies pending fees.

Report Management:

The system generates reports related to students, faculty, attendance, admissions, and fees.

Notification Management:

The system provides notifications and alerts for important administrative activities.

1.4 Expected Outcomes

The expected outcomes of the system are:

- Faster college administrative operations.
- Centralized management of college information.
- Reduction in paperwork.
- Improved data accuracy.
- Easy access to student and faculty information.
- Efficient attendance and fee management.
- Improved communication and reporting.
- Secure and organized data storage.
- Easy deployment using Docker.

2. Project Structure Design

The proposed folder structure of the College Administration and Management System is:

College-Administration-Management/

```
|  
|— frontend/  
| |— public/  
| |— src/  
| | |— components/  
| | |— pages/
```

```

| | | └── services/
| | | └── App.js
| | └── package.json
|
└── backend/
    | └── controllers/
    | └── models/
    | └── routes/
    | └── middleware/
    | └── config/
    | └── server.js
    | └── package.json
    |
    └── database/
        | └── schema.sql
        |
        └── docs/
            | └── architecture.png
            | └── workflow.png
            |
            └── screenshots/
                |
                └── Dockerfile
                └── docker-compose.yml
                └── .gitignore
                └── .env
                └── README.md

```

Purpose of Major Folders and Files

Folder/File	Purpose
-------------	---------

frontend	Contains the user interface of the application
backend	Contains server-side logic and APIs
controllers	Handles application operations
models	Defines database models
routes	Defines API endpoints
middleware	Handles authentication and request processing
config	Contains application configuration
database	Contains database schema and database-related files
docs	Stores project documentation and diagrams
screenshots	Stores screenshots of Git and Docker operations
Dockerfile	Contains instructions for creating the Docker image
docker-compose.yml	Defines and manages application containers
.gitignore	Specifies files that should not be tracked by Git
.env	Stores environment-specific configuration
README.md	Contains project description and setup instructions

3. Git Repository Initialization

Git is used as the version control system for the project. It tracks changes, maintains the project history, and supports collaborative development.

Git Initialization

The repository can be initialized using the following commands:

```
mkdir College-Administration-Management
```

```
cd College-Administration-Management
```

```
git init
```

```
git status
```

```
git add .
```

```
git commit -m "Initial project setup"
```

```
git branch -M main
```

To connect the local repository with GitHub:

```
git remote add origin <repository-url>
```

```
git push -u origin main
```

Essential Git Files

.git folder:

The .git folder is created when the repository is initialized. It stores Git configuration, branches, commit history, and other repository information.

.gitignore:

The .gitignore file prevents unnecessary or sensitive files from being uploaded to the repository.

Example:

```
node_modules/
```

```
.env
```

```
*.log
```

```
dist/
```

```
build/
```

README.md:

The README file provides information about the project, technologies used, installation steps, and execution instructions.

4. Git Branching Strategy

The project uses a structured branching strategy to support collaborative development.

Branches

Main Branch:

Contains stable and production-ready code.

Development Branch:

Contains the latest integrated development code.

Feature Branch:

Used to develop individual features such as student management, faculty management, and fee management.

Release Branch:

Used for final testing and preparation before releasing a new version.

Hotfix Branch:

Used to fix critical errors in the production version.

Branching Workflow

Main

|



Development

|

|— Feature/Student

|— Feature/Faculty

|— Feature/Admission

|— Feature/Fee

|



Development

|



Release

|



Main

This branching strategy prevents unstable development work from directly affecting the main branch and allows different developers to work on different features independently.

5. Version Control Workflow

The Git workflow followed in the project is:

Create Feature



Create Feature Branch



Develop and Test



Git Add



Git Commit



Git Push



Merge with Development



Testing



Release



Main Branch

Important Git Commands

Create a feature branch:

```
git checkout -b feature/student-management
```

Check the repository status:

```
git status
```

Add files:

```
git add .
```

Commit changes:

```
git commit -m "Add student management module"
```

Push the branch:

```
git push -u origin feature/student-management
```

Switch to development:

```
git checkout develop
```

Merge the feature:

`git merge feature/student-management`

Download the latest changes:

`git pull origin develop`

Push changes:

`git push origin develop`

Conflict Resolution

If a merge conflict occurs, the conflicting files are identified using:

`git status`

After manually resolving the conflicts:

`git add .`

`git commit -m "Resolve merge conflict"`

`git push`

6. Commit History Analysis

Meaningful and descriptive commit messages are maintained throughout the project.

Example commit history:

Initial project setup

Add student management module

Add faculty management module

Implement admission management

Add attendance module

Implement fee management

Add database configuration

Create Dockerfile

Add Docker Compose configuration

Fix authentication issue

Update project documentation

Good commit messages make the project history easy to understand and help developers identify changes, features, and bug fixes.

The commit history can be displayed using:

```
git log --oneline --graph --all
```

Screenshot:

Insert screenshot of the git log --oneline --graph --all command here.

7. Docker Environment Design

Docker is used to containerize the College Administration and Management System. Containerization ensures that the application and its dependencies run consistently across different environments.

Why Docker is Suitable

Docker provides:

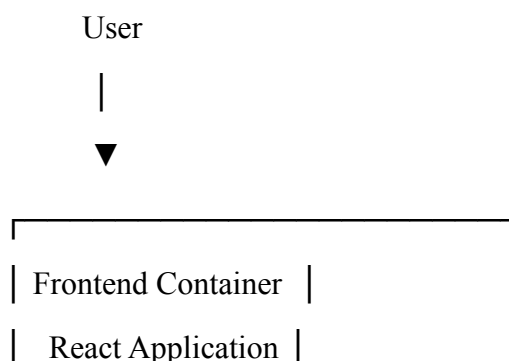
- Application portability.
- Consistent development environments.
- Dependency isolation.
- Easy deployment.
- Faster application setup.
- Better scalability.
- Simplified maintenance.

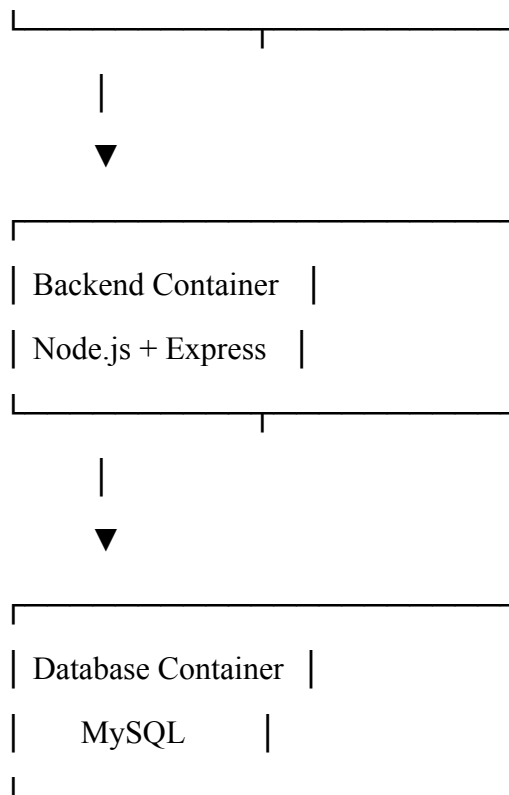
Components Requiring Containerization

The application can be divided into the following components:

1. Frontend Container
2. Backend Container
3. Database Container

Docker Architecture





8. Dockerfile Development

A Dockerfile contains the instructions required to create a Docker image for the application.

Dockerfile

```
FROM node:20
```

```
WORKDIR /app
```

```
COPY package*.json ./
```

```
RUN npm install
```

```
COPY . .
```

```
EXPOSE 5000
```

CMD ["npm", "start"]

Explanation of Dockerfile Instructions

FROM:

Specifies the Node.js base image.

WORKDIR:

Creates and sets the working directory inside the container.

COPY:

Copies the package files and application source code into the container.

RUN npm install:

Installs all required project dependencies.

EXPOSE:

Specifies the port used by the application.

CMD:

Starts the application when the container is executed.

9. Docker Compose Design

Docker Compose is used to define and manage multiple containers as a single application.

docker-compose.yml

services:

backend:

build: ./backend

ports:

- "5000:5000"

environment:

DB_HOST: database

DB_USER: root

DB_PASSWORD: password

DB_NAME: college_db

depends_on:

- database

database:

image: mysql:8

environment:

MYSQL_ROOT_PASSWORD: password

MYSQL_DATABASE: college_db

ports:

- "3306:3306"

volumes:

- mysql_data:/var/lib/mysql

volumes:

mysql_data:

Explanation

Backend Service:

Runs the Node.js and Express backend application.

Database Service:

Runs the MySQL database.

Ports:

Allow communication between the host system and containers.

Environment Variables:

Provide database configuration details.

Depends_on:

Defines the dependency between the backend and database services.

Volumes:

Ensure that database data is preserved even when containers are restarted or recreated.

10. Container Deployment and Testing

The application is deployed using Docker Compose.

Build Docker Images

docker compose build

Start the Containers

docker compose up

Run in Background

docker compose up -d

Check Running Containers

docker ps

View Container Logs

docker compose logs

Stop the Containers

docker compose down

Testing Procedure

The following testing procedure is followed:

1. Build the Docker images.
2. Start the application containers.
3. Verify that the backend container is running.
4. Verify that the database container is running.
5. Open the application in a web browser.
6. Test the administrator login.
7. Add and update student records.
8. Test faculty management.
9. Test admission management.
10. Test attendance and fee management.
11. Verify database connectivity.
12. Check whether the application functions correctly.

Screenshots

Screenshot 1: Docker image build.

Screenshot 2: Docker Compose execution.

Screenshot 3: Running containers using docker ps.

Screenshot 4: College Administration application running in the browser.

11. Benefits of Git and Docker

Benefits of Git

Collaboration:

Git allows multiple developers to work on the same project using separate branches.

Version Management:

Git maintains the complete history of project changes.

Branching:

Different features can be developed independently without affecting stable code.

Backup:

The remote repository provides a backup of the project source code.

Conflict Resolution:

Git provides mechanisms to identify and resolve conflicts between different changes.

Benefits of Docker

Portability:

The application can run consistently on different systems.

Isolation:

Application components and their dependencies are isolated inside containers.

Deployment:

Docker simplifies application deployment.

Scalability:

Containers can be replicated when additional resources are required.

Maintainability:

Application dependencies and configurations are organized and controlled.

12. Challenges and Improvements

Challenges Faced

The major challenges during implementation include:

- Managing multiple Git branches.
- Resolving merge conflicts.
- Maintaining meaningful commit messages.
- Configuring Docker containers.
- Establishing communication between backend and database containers.

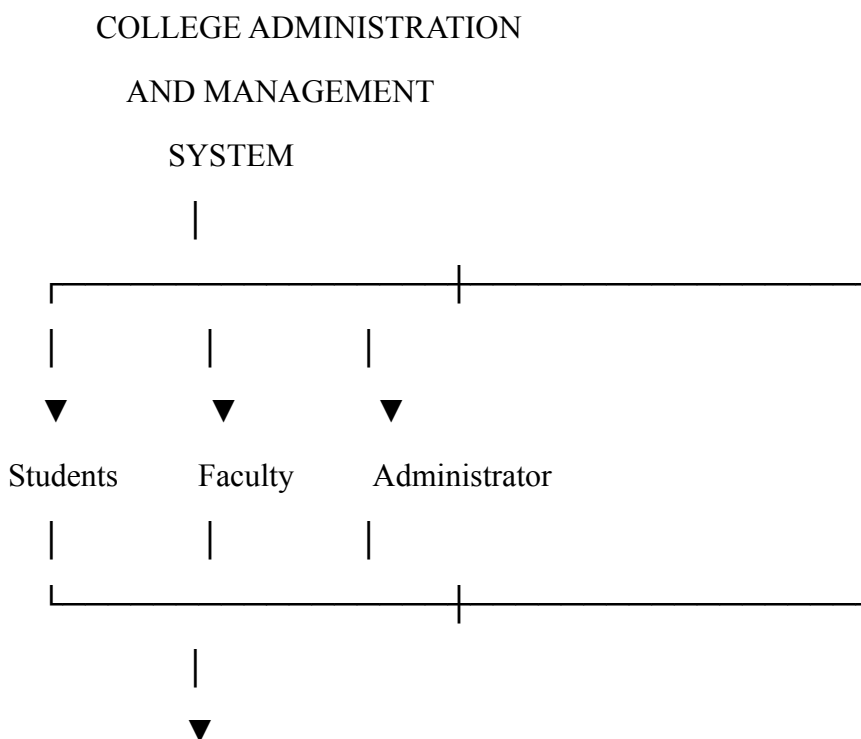
- Managing environment variables.
- Maintaining persistent database storage.
- Debugging Docker-related errors.

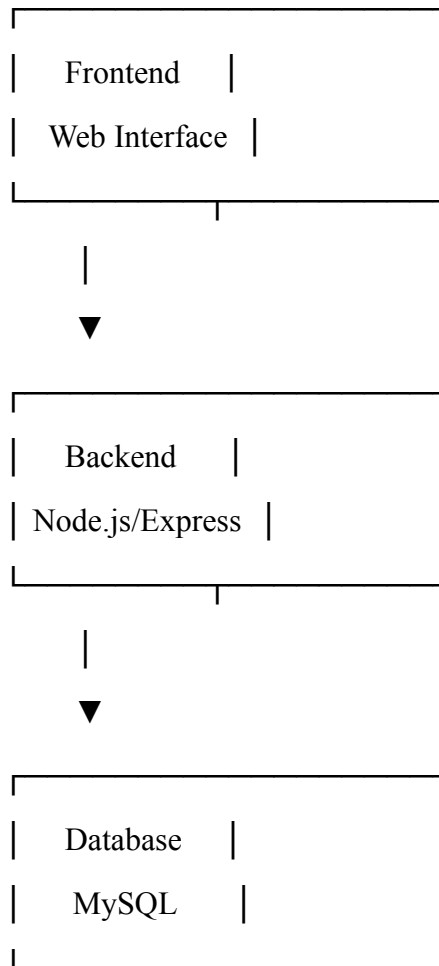
Future Improvements

The following improvements can be implemented in the future:

- Implement role-based authentication for administrators, faculty, and students.
- Add an interactive administration dashboard.
- Implement automated testing.
- Integrate CI/CD using GitHub Actions.
- Use Docker multi-stage builds.
- Add application monitoring and logging.
- Deploy the system to a cloud platform.
- Implement automated database backups.
- Improve security using encrypted credentials.
- Add email and notification services.

System Architecture





Major System Modules

College Administration and Management System

- |
- |— Student Management
- |— Faculty Management
- |— Department Management
- |— Course Management
- |— Admission Management
- |— Attendance Management
- |— Fee Management
- |— Examination Management
- |— Notification Management
- |— Reports and Analytics

Conclusion

The **College Administration and Management System** provides a centralized solution for managing important college administrative activities. Git provides effective version control, collaboration, branching, and change management, while Docker provides portability, dependency isolation, and simplified deployment. The combination of Git and Docker improves the overall maintainability, reliability, scalability, and deployment efficiency of the application.